Data Mining Techniques A Real Life Competition. Group 10.

Toni Kenđel[1 [2797438]], Kenia Lopez Sotomayor[1 [2682153]], and Iñigo Auza de la Mora[1 [2824537]]

Vrije Universiteit Amsterdam, 1081 HV Amsterdam, The Netherlands

Keywords: Data Mining · Neural Network · Machine Learning.

1 Introduction

As an illustration of Data Mining, this report details the procedures taken to approach a data set containing information about a search of a user for a hotel and the hotels properties. The goal of this project is to analyze the data and train a model that will be able to predict a list of the most likely to be clicked on hotels depending on the user query. Each user in the data set has multiple rows of data provided. In the following sections we present our evaluation of the data and the processing applied to prepare it for four prediction models.

2 Business Understanding

Travelers with different interests prioritize various aspects when searching for a hotel. However, certain attributes are commonly valued by most travelers. Studies have indicated that features such as *value* (price), *location*, *sleep quality*, *rooms*, *cleanliness*, and *service* are significant, with *rooms* and *value* being the most important overall. When comparing low-rated and high-rated hotels, *rooms* followed by *cleanliness* are the most crucial attributes for low-rated hotels. Conversely, for high-rated hotels, *cleanliness* is the top priority, followed by *service*. Notably, *value* and *location* are not among the top features in either case. [3].

2.1 Related work

Understanding of the hotel industry combined with advanced data analysis and machine learning techniques enables companies such as Expedia to predict which hotel properties users are most likely to click on. Competition in Kaggle has enabled users to work on such predictions. The variables used as predictors varies per user, but also by type of model used. For instance, participants from past year [1], when using Pairwise Logistic Regression, used only seven features, which were *srch_ id*, *prop_ id*, *srch_ destination_ id*,*prop_ starrating*, *prop_ location_ score1*, *prop_ location_ score2*, *price_ usd*. While when using Gradient Boosting Machine

and Extreme Randomized Trees, they applied feature engineering to create new features. Those new feature were mainly features with ratings and prices. To name a few

$$starrating_diff = visitor_hist_starrating - prop_starrating$$
$$price_diff = visitor_hist_adr_usd - prop_starrating$$
$$total_fee = price_usd * srch_room_count$$
$$per_fee = \frac{price_usd * srch_room_count}{srch_adults_count * srch_children_count}$$

Another person joining past competitions created a subset of the raw numerical variables, which included $room_price$, $review_score$, $star_rating$, and also all the competitor rates and search attributes. ID features were no used since they did not provided discriminatory information. Furthermore, they also created new features combining two different features [2].

3 Data Preparation

3.1 Exploratory Data Analysis

The data consisted on:

Total number of features: 53.
Total number of user queries: 4958347.
Total number of searches: 199795.

The data consists of many features that describe specific queries that different users have received based on their search terms, see Table 2. From the dataset we can obtained massive information, for instance, the number of different hotels, destinations that user are interested on, etc, Table 1. The resulting data contains many imbalances between queries that have been clicked on and ones that have not been clicked on where a total of 221879 queries have been clicked. This number shrinks further once we look at the hotels that have been booked. This can be further seen in Fig 1. The data is indicating that out of the many presented options that Expedia is trying to provide to its customers the likelihood of them actually clicking on a provided result is 4.5%. This can be due to various reasons, financial, social or other unknown effects that could determines if the user will click on a given hotel.

Table 1: Some statistics of the training dataset.

Groups	
Hotels	129113
Customer location	210
Country destination	172
Locations in destination	18127

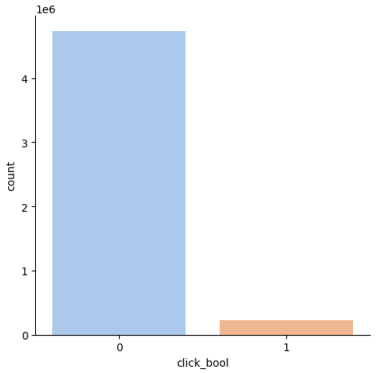
Table 2: Overview of the training dataset. Only a few variables are shown.

Search query			
search id	date time	room count	adults count
1	2013-04-04 08:32:15	1	4
1	2013-04-04 08:32:15	1	4
...	...	...	...
1	2013-04-04 08:32:15	1	4
1	2013-04-04 08:32:15	1	4

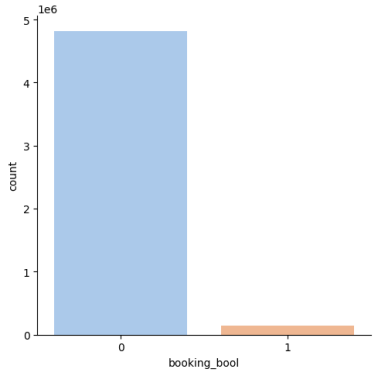
Price competitiveness		
comp 1 rate		comp 8 inv
NA	...	0
NA	...	0
...	...	...
NA	...	0
NA	...	0

Property characters				
prop id	review score	promotion flag	position	price USD
893	3.5	0	27	104.77
10404	4	0	26	170.74
...	...	...	...	...
131493	4.5	0	29	334.00
133689	4.5	0	12	279.00

Target	
click bool	booking bool
0	0
0	0
...	...
0	0
0	0



(a) Distribution of not clicked (0) vs clicked queries (1).



(b) Distribution of not booked (0) vs booked queries (1).

Fig. 1: Graphical representation of disproportion of label features in the data set.

The given dataset contained 31 features with missing values, specifically, $visitor_hist_starrating$, $visitor_hist_adr_usd$, $prop_review_score$, $prop_location_score2$, $srch_query_affinity_score$, $orig_destination_distance$, $gross_bookings_usd$, $comp_i_rate$, $comp_i_inv$, $comp_i_rate_percent_diff$, where i is the number of competitor from 1 to 8.

Looking at the possible correlations between the different features in the data set we used a Spearman's correlation matrix which can be seen in Fig. 2. Here we

can see the relation between *click_ bool* and *booking_ bool* which is to be expected given that one can't be without the other, and also *gross_ booking_ usd* seems to be in high correlation with the two, however this feature will be dropped since together with the feature *position* they are not appearing in the test set. Looking at *click_ bool*, 3rd to last, and *booking_ bool* at the bottom of the matrix and their relations to the rest of the data we can see that *prop_ location_ score2*, *promotion_ flag* and *prop_ review_ score* seem to have a slight correlation however its nothing big. Interestingly, the competitor features seem to be very correlated between them, seeing the cluster formed on the bottom right of the heatmap.

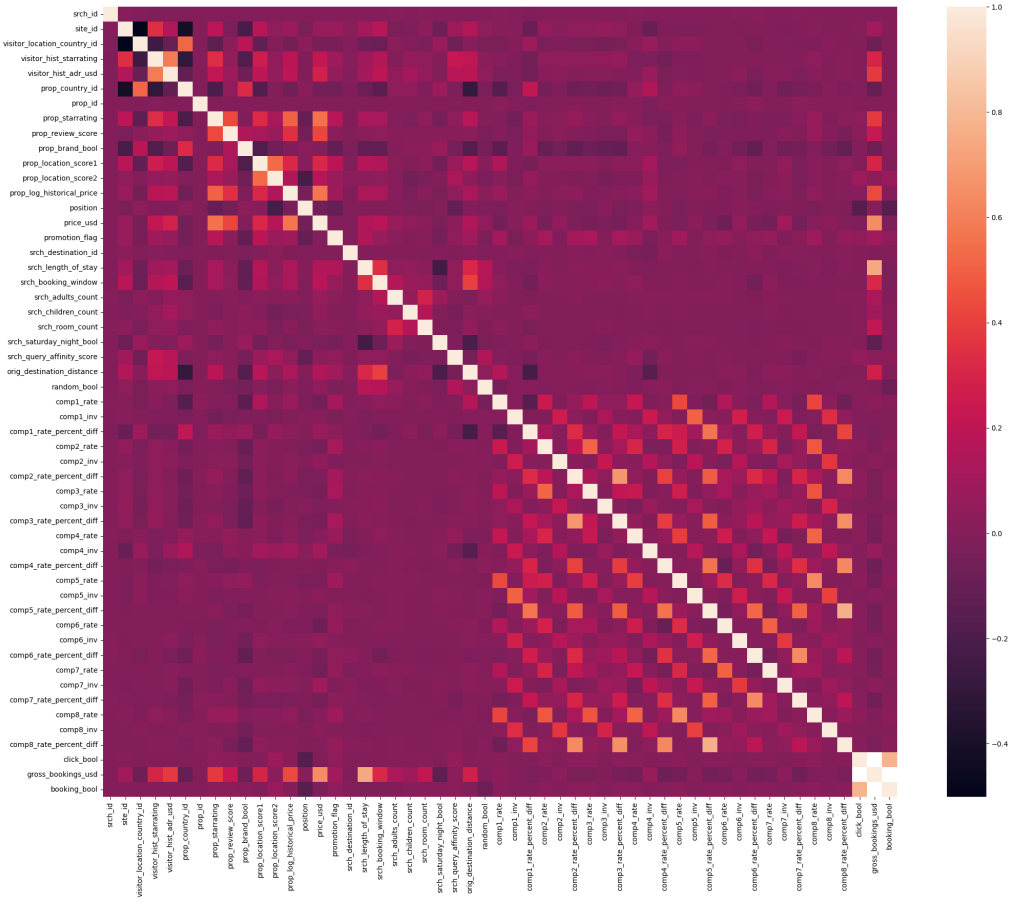


Fig. 2: Spearman correlation between variables in the training dataset.

Looking at this data further, we can notice that the data is highly skewed towards a specific country to which the queries are being catered to. This can

be seen in Fig. 3. Given the currency that is being also used at large we believe that the data is largely based on queries from people of the USA.

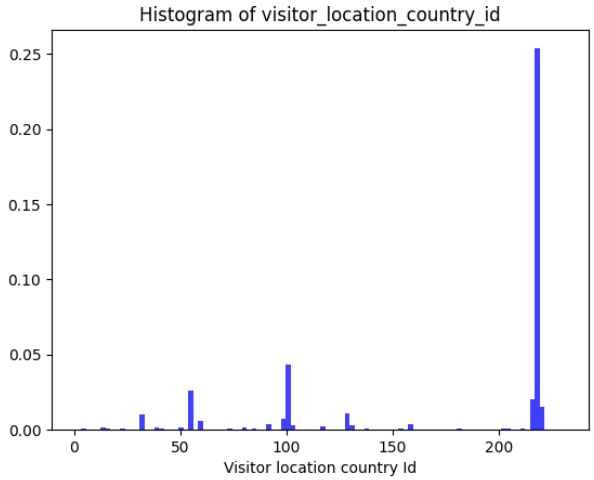


Fig. 3: Histogram showing the percentage of visitors per country ID. One country, which believed is people from USA, has the highest amount of queries.

3.2 Data Cleaning

Outliers and missing values can have an impact on how the data influences the training of a model. For that reason we have tried several approaches with varying effects.

Fill NaN approach
We followed multiple approaches to impute missing values that deal with time series; fill NaN with 0, fill NaN with mean, fill NaN with median.
The first approach was done in order to create a quickly usable data set for getting a general score out of a model. This method would proceed to change all NaN values in data set to the value 0. Surprisingly this method proved to be the best performing one once we started training the model and we decided to keep this procedure for dealing with NaN values.

The second approach of filling NaN values with the mean worked in a similar fashion to the previous fill function, where instead, the empty cells were filled with the mean of the feature value. We have tried performing this on the feature *srch_query_affinity_score* however this would lead in minor decrease in performance.

The third approach of filling with the median of the feature value was performed on features like *orig_destination_distance* but it also resulted in a decrease in performance.

We have attempted using filling of NaN with a value of 3 for features such as *prop_review_score* where the review score is between 0 and 5 giving it a middle score to all empty cells. Again this lead to a decrease in performance so it was replaced with the first approach.

Data type conversion
We noticed that the boolean features came cast to type int64 in the raw dataset so we have decided to convert some of them back to a categorical value. The features that have been converted have been all id and bool features together with features such as *promotion_flag, srch_booking_window* and newly added features like *room_per_person, same_country* which we go over in subsection 3.3.

Outliers
Given that we decided to use an algorithm based on an ensamble of decision trees, we were not to concerned with dealing with them sine the algorithms are quite resilient to outliers.

3.3 Feature Engineering

New variables

Obtaining new features that represent more precisely our underlying problem, i.e. predicting which hotel that result from a search of a user, the user is most likely to click on, will play a big role in our predicting model.

During our research, several linear models and generalized linear models were used to search for a level of relation between the features and the click and booking tags, individually. To achieve this, the variables $comp_i_rate$ were cleaned, replacing missing values with 0. This reflected the unknown or neutral comparison, while -1 was a disadvantageous and 1 advantageous. For this, the individual $comp_i_rate$ variables, and different methods of recombination were tested. These included the summation of the factors, utilizing different scales, normalization, and combination of the previous. Many of them showed to have similar level of influence, or were correlated. After verifying the influence of the combined variables on the models ability to predict, the final *comp* variable was made by combining the entries of all $comp_i_inv$ variables, summing the values and NaN entries being replaced by 0.

Lastly, an Exhaustive Search was utilized to reduce the number of variables. Due to time constrain, an Exhaustive Search with all parameters would have taken too much time and computing power. Therefore, the Exhaustive Searches were made in smaller batches, reaching the conclusion of the *comp* variable's importance, but suggesting the need for $comp_2_rate$, $comp_3_rate$, and $comp_8_rate$ to improve accuracy. The process for selecting the features for

the $comp_i_inv$ variables was very similar. However, the results showed the importance of $comp_8_inv$, and the newly created inv, which was made similarly to $comp$ through the sum of all $comp_i_inv$ values.

We also attempted to create new features following the formulas defined by a past competitor [1] for their Gradient Boosting Machine model, defined also in subsection 2.1. However, as the model grew more complex, we designed new features to better fit our training. These are distinguished into two categories: $features$ and $targets$. Features aided the accuracy of the model, and were meant to increase the information to which the model had access to.

New Features

- *comp*: sum of all $comp_i_rate$ variables with the missing values replaced by 0.
- *inv*: sum of all $comp_i_inv$ variables with the missing values replaced by 0.
- *room_minus_guess*: number of rooms requested minus total sum of adult and children guests.
- *room_per_person*: boolean, 1 if there is a room per person, 0 otherwise.
- *room_guest_ratio*: total sum of adult and children guests, over the total number of rooms.
- *price_usd_norm*: normalized usd price.
- *same_country*: boolean, 1 if the guest's and property's country ID match, 0 otherwise.

Meanwhile, the target features were used to train models to predict different goals. These features on their own are not very useful, as training a model on them would cause it to predict the wrong target. However, as it will be detailed in the *Algorithm Implementation* section, they can be used to train models, whose predictions can be fed into a further model. An important note is that for the chosen LightGBM model, the target were categorized, thus needed to be labels and not a continuous range.

New Target Features

- *superscore*: sum of *booking_bool* x 4 + *click_bool*.
- *lbl_ratio_booked_vs_clicked*: Normalized ratio of total bookings over total clicks on that property; multiplied by 10 and rounded to 0 decimals, to create integer labels from 0 to 10.
- *lbl_ratio_booked_vs_count*: Normalized ratio of total bookings over appearances of that property; multiplied by 10 and rounded to 0 decimals, to create integer labels from 0 to 10.
- *lbl_ratio_clicked_vs_count*: Normalized ratio of total clicks over appearances of that property; multiplied by 10 and rounded to 0 decimals, to create integer labels from 0 to 10.
- *lbl_ratio_superscore_vs_count*: Normalized ratio of individual *superscore* over appearances of that property; multiplied by 10 and rounded to 0 decimals, to create integer labels from 0 to 10.

In retrospective, the increase in accuracy most of the new features contributed was rather small. It is possible that different approaches, or a larger variety of new features would have resulted in a higher accuracy. The reasons for their inclusion and implementation will be discussed in the next section.

4 Algorithm Implementation

LightGBM
In order to rank the properties belonging to a user search on the likeliness that the property will be booked, we use the LightGBM framework, using a gradient boosting technique. This method relies on decision tree algorithms and is applied for tasks such as ranking, classification, and various other machine learning assignments.

The way in which the data is prepared for training has an additional requirement which distinguish them from other models. This consists on an additional array containing the number of queries per user, *srch_id*. During training, the LightGBM model utilizes this information to optimize its learning process and effectively capture user search patterns and preferences. This allows the model to learn from the ranking in the group context they were assigned.

LightGBM characterizes for implementing leaf-wise (vertical) tree growth for different tasks, like classification, obtaining great loss reduction, leading to improved accuracy and faster processing speeds. Some of the important parameters of this models are *max depth*, *number of leaves*, *learning rate*, among others. Since we are mainly focused on ranking, *lamdarank* or *rank xendcg* are the objective functions to be used.

LightGBM - Single Layer Model
Simultaneously, we developed several models with slight different techniques. The chosen metrics to estimate the accuracy of the models before submitting to Kaggle were Mean Average Precision (MAP) and Normalized Discounted Cumulative Gain (NDCG). While both MAP and NDCG are effective at estimating values for groups and classes in ranking tasks [6], the NDCG evaluation metric approached better that of the Kaggle competition.

At the early stages of the model, the features *srch_id*, *prop_id*, $comp_8$*_rate*, $comp_8$*_inv*, *price_usd*, *prop_log_historical_price*, *prop_review_score*, *prop_starrating*, and *comp* were utilized. These were a varied selection of features meant to serve as guideline during the implementation of the system, rather than pushing for accuracy. These included $comp_8$*_rate* and $comp_8$*_inv*, which our research during feature engineering had shown to have some influence; *srch_id* which was needed to train the model in groups; *comp* which was the only engineered feature at the moment of testing; and other varied features that were thought to maybe have an important influence, yet were present mainly for variety. The private Kaggle score of the test dataset was 0.31922. All private and public Kaggle scores can be found in table 5.

It is important to notice that the features that were categorical, were marked as so, passing them as type 'category' to the data frame before training.

After successfully generating a model, we opted to base our features on the research done by Liu, X., Bing, X., Bin, W., in a similar task [1]. We were hopping to create a baseline for guidance. However the model with these new features under performed, receiving a 0.26616 private score.

Following, we attempted to pass all features present in both the training and test set, as the model requires a matching set of features. This was done again with the hopes of setting a baseline. This time, the model improved drastically, receiving a private score of 0.38826.

Before continuing to engineer more features, the team attempted to fine-tune the parameters of the model. The maximum number of bins for categorization was increased to 500. This helped the model better categorize the values. However, other larger bins would create over fitting and decrease accuracy. Additionally, the number of estimators was increased, reaching the 5000 estimators at a given time, while the learning rate decreased to 0.01 and lower. This resulted in longer training times, and unfortunately resulted in no improvement.

Given the time constrain and computing power available, the following models were trained with 100 estimators, a learning rate of 0.1 and a max bin size of 500. This was due to prioritize development of features and shorten training times.

The best single model this approach yielded received a 0.39179 private score.

LightGBM - Cumulative Model

The following step taken to seek a higher score was to create a multi-layer model. There were two models that utilized this technique. Each of them utilized the same first layer models, which then fed the estimation into the second layer models.

Each of these first layer models were variations of Lambdarank LightGBM models. Many of them had different target features, thus learning different aspects of the data. For training, they received all the features shared between the train and test sets, in addition to the newly created features, in the same way the previously mentioned LightGBM model did. Initially, it was hoped that we could adjust the features of every model to better fit their training, but the much longer train time and the time constraint of the project made this difficult. The adjustments made to each model can be seen in table 3.

The first of the cumulative models utilized a second layer model that took features *srch_id*, *prop_id*, and the predictions of the previous 9 models. This model received a private score of 0.39092. Although not better than the best of the single layer LightGBM models, it was fairly close. More importantly, it showed that the model was able to retain information through multiple layers.

The last of the cumulative models utilized a second layer model that, like the previous one, utilized the predictions of the 9 first layer models, but in addition utilized all of the features fed into those first layer models. This was due with the hope of gaining some insight in the relation and pre-calculation of the first

layer models, while maintaining all the information from the original features. The parameters of these two models can be seen in table 4.

This last model achieved our highest score, with a private score of 0.39198. Nonetheless, the much larger computational expense is hard to justify, specially given the 0.39179 score of the single layer LightGBM. This, however, will be discussed in the following sections.

Table 3: First layer models

Model #	Target	Testset size	# Estimators	Learning rate	Max bin size	Features	Data order
1	*superscore*	0.001	100	0.1	500	All	Original
2	*superscore*	0.1	100	0.1	500	All	Inverse
3	*lbl_ratio_booked_vs_clicked*	0.2	100	0.1	255	All	Original
4	*superscore*	0.001	100	0.1	500	All	Inverse
5	*click_bool*	0.001	100	0.1	500	All	Original
6	*booking_bool*	0.001	100	0.1	500	All	Original
7	*lbl_ratio_booked_vs_count*	0.001	100	0.1	500	All	Inverse
8	*lbl_ratio_clicked_vs_count*	0.001	100	0.1	500	All	Original
9	*lbl_ratio_superscore_vs_count*	0.001	100	0.1	500	All	Original

Table 4: Second layer models

Model #	Target	Testset size	# Estimators	Learning rate	Max bin size	Features	Data order
1	*superscore*	0.01	100	0.1	500	first layer model predictions	Original
2	*superscore*	0.01	100	0.1	500	first layer model predictions and all first layer model features	Original

5 Evaluation

The scores of a selected group of the submitted models can be seen in table 5. The scores are the calculated "Normalized Discounted Cumulative Gain (NDCG)@5 calculated per query and averaged over all queries with the values weighted by the log_2 function" [4].

From the table we can notice how the choice of features, and the newly created ones, have an important impact in the accuracy of the model. However, you may also notice how quickly the progress reached a level of stagnation.

Table 5: Kaggle scoring of the test dataset

Model	Public score	Private score
Initial features	0.31832	0.31922
Researched features	0.26496	0.26616
Initial and created features	0.38627	0.38826
Initial and created features (highest)	0.38964	0.39179
Cumulative model 1	0.38944	0.39092
Cumulative model 2	0.38983	0.39198

As we developed our models, we quickly realized how much more complicated is to utilize the recorded data and generate new meaningful data that contributes to the prediction of the model. In the larger scale of things, the difference in score among the three best performing models is unlikely to be significant if applied to a different test dataset.

On the other hand, the accuracy of our model greatly outperforms a random order. Although the performance may not be at the par with other exemplary deployed ranking systems, given the time we had to implement the model, it seems fair to say that our system performs adequately.

In order to improve our model, further development of features would be beneficial. Additionally, although the single layer LightGBM model received parameter tuning (see section 4 for further details), the cumulative models did not. Some of the parameter tuning of the single layer model of course is carried on when training the first layers, but each of them could be further tuned to the specific goal they were targeted. This includes a stricter selection of features used, and a longer training time with a lower learning rate.

A limitation of this model is interpretability, as it is not explicitly obvious what features cause a specific selection. This is specially true with the growing number of features and categories.

Additionally, it is often not easy to predict how the system will behave with a new test set. The accuracy can sometimes change drastically. Adding to the challenge, is the human factor. Trends and differences in changing generations, economical fluctuation, among many, affects the behaviour of the clients. This means that, however accurate the system is, not only can the clients behave differently, but the selected features could become obsolete. This is always the risk with a changing user base.

6 Scalable Deployment

The question of scalability is one that requires the consideration of multiple factors. A more accurate model often requires a longer training time. However, if accuracy is the priority, longer training times may be adequate. This is specially true if the model does not need to be retrained often.

Considering that the best performing single-layer LightGBM model takes only a couple minutes, retrainability does not seem to be a concern. In contrast, the cumulative models take a little over half hour, when ran in the same computer. This could mean that adjusting the model after deployment, which could take several tries, would take considerably longer. During that time, the live service will continue to be running with a faulty model. Please be aware that although this is a fair comparison as the models were ran under the same circumstances and equipment, this is not an appropriated measure of the computing power required to train the model. This runtime could become irrelevant in a more capable machine.

In the other hand, given the enormous flow of traffic that large booking websites have, accuracy could be of greater importance. The 0.2% of 5,000,000

visits, remains a large number (10,000). Given that order of the presented options is very relevant to the choosing of the items [5], in this case the properties, accuracy may play a much more important factor. Receiving feedback during deployment may be able to show some insight into this.

The scalability of the model can be condensed to: can it be retrained fast enough, given the training time and times it needs to be retrained in a certain period?; or is higher accuracy, with a higher training cost, more relevant to the market? This will depend on how much the behaviour of the clients fluctuates, how long adjusting a model takes (including training), and how long this model is good for.

References

1. Liu, X., Bing, X., Bin, W.: Combination of diverse ranking models for personalized expedia hotel searches. arXiv preprint (2013)
2. Thomas Belluci, Hu Qingzhi, Lin Chih-Chieh. (2019). On the Ranking of Expedia Search Results using LambdaMART. https://chihchiehlin.com/documents/DMT_2_group55.pdf
3. Rhee HT, Yang SB. (2015). Does hotel attribute importance differ by hotel? Focusing on hotel star-classifications and customers' overall ratings. Computers in Human Behavior. 50(3), pp.576-587.
4. Bachflo, Kaggle user (2024). DMT 2024 2nd assignment, 2024 VU Data Mining Techniques Cup. https://www.kaggle.com/competitions/dmt-2024-2nd-assignment/overview
5. Floris den Hengst (2024).Recommender Systems. X400108 Data Mining Techniques, Vrije Universiteit Course.
6. Ransaka Ravihara (2023). How to Implement Learning to Rank Model using Python. https://github.com/Ransaka/LTR-with-LIghtGBM https://towardsdatascience.com/how-to-implement-learning-to-rank-model-using-python-569cd9c49b08